

Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 «Математика и компьютерные науки»

Отчёт о выполнении курсовой работы

По дисциплине: «Теоретические основы баз данных»

На тему: «Учёт компьютерных шрифтов электронной типографии»

№ этапа	Дата	Подпись
1		
2		
3		

Выполнил студент
группы 5130201/40002

_____ Семенов И. А.

Проверил
преподаватель

Попов С. Г.

«_____» 2026г.

Санкт-Петербург
2026

Содержание

1	Аналитика	3
1.1	Описание предметной области «Учёт компьютерных шрифтов в электронной типографии»	3
1.2	Цели функционирования базы данных	6
1.3	Польза от внедрения базы данных	7
1.4	Описание сущностей	7
1.5	Описание ролей и субъектов	8
1.6	ER-диаграмма	9
1.7	Схема объектов	11
2	Проектирование	12
2.1	Лингвистическое описание схемы базы данных	12
2.2	Описание таблиц базы данных	13
2.3	Создание таблиц базы данных и генерация записей	21
3	Программирование	24
Приложение А. Исходный код SQL-скрипта для создания таблиц базы данных		25
Приложение Б. Программа на TypeScript для заполнения базы данных тестовыми данными		27

1 Аналитика

1.1 Описание предметной области «Учёт компьютерных шрифтов в электронной типографии»

Типографика — искусство оформления текста с помощью наборного шрифта — является одним из фундаментальных элементов визуальной коммуникации — передачи информации посредством зрительно воспринимаемых образов. На протяжении столетий она развивалась от ручного набора металлических литер (брусков с выпуклым изображением знака) до современных цифровых технологий. Изобретение печатного станка Иоганном Гутенбергом в середине пятнадцатого века положило начало массовому распространению печатного слова, однако процесс набора текста оставался трудоёмким ремеслом. Каждая литера отливалась из металла, а наборщик вручную составлял строки из отдельных знаков. Появление линотипа (машина, отливающая целые строки) и монотипа (машина, отливающая отдельные литеры) в конце девятнадцатого века механизировало этот процесс, но подлинная революция произошла с переходом к цифровым технологиям.

Электронная типография — организация, осуществляющая подготовку и выпуск печатной и цифровой продукции с использованием компьютерных технологий, — представляет собой современный этап развития печатного дела, в котором все процессы подготовки и воспроизведения текста осуществляются с использованием компьютерных технологий. В отличие от традиционной типографии, где шрифт существовал как физический объект, в электронной типографии шрифт является программным продуктом. Компьютерный шрифт — это файл, содержащий векторные или растровые описания графем (букв, цифр и других знаков письма), а также метрическую информацию, необходимую для корректного отображения и печати текста. Такие шрифты используются в издательских системах, графических редакторах, веб-страницах и пользовательских интерфейсах операционных систем.

В наши дни современную электронную типографию невозможно представить без обширных коллекций шрифтов. Дизайнеры, верстальщики и препресс-специалисты (специалисты по допечатной подготовке) ежедневно работают с десятками гарнитур, подбирая оптимальные решения для каждого проекта. Шрифтовая гарнитура объединяет набор начертаний, выполненных в едином стилистическом ключе. Начертания различаются насыщенностью штрихов (толщиной линий знака) — от сверхтонкого до сверхжирного — и наклоном знаков, формирующим прямое, курсивное (с изменённым рисунком букв) или наклонное (линейно наклонённое прямое) написание. Одна гарнитура может насчитывать от единственного начертания до нескольких десятков вариантов, что позволяет

выделять структурные элементы документа — заголовки, подзаголовки, основной текст.

Шрифты классифицируются по конструктивным особенностям строения знаков. Антиквенные шрифты, также называемые серифными (*от serif*), имеют характерные засечки на концах штрихов. Гротески, или рубленые шрифты (*sans serif fonts*), лишены засечек и отличаются геометрической простотой форм. Моноширинные шрифты (*monospace fonts*) характеризуются одинаковой шириной всех знаков, что делает их незаменимыми в программировании и табличной вёрстке. Акцидентные шрифты предназначены для заголовков, логотипов и декоративных целей, тогда как рукописные имитируют различные виды ручного письма. Шрифты также характеризуются стилистическими особенностями: геометрические построены на основе простых форм, гуманистические отличаются плавными формами, имитирующими ручное письмо пером, брусковые отличаются прямоугольными засечками и одинаковыми толщинами со штрихами.

Важной характеристикой компьютерного шрифта является поддержка языков и письменностей. Глифовый (*от glyph, графическое представление символа*) состав определяет, какие алфавиты способен отображать шрифт. Базовая латиница охватывает западноевропейские языки, расширенная латиница добавляет знаки для центральноевропейских и балтийских языков, кириллица обеспечивает поддержку славянских языков. Системы письма различаются также направлением чтения: латиница и кириллица читаются слева направо, арабское и еврейское письмо — справа налево, традиционные восточноазиатские тексты могут располагаться вертикально сверху вниз. Направление письма влияет на технические требования к отображению шрифта и должно учитываться при организации шрифтовой коллекции.

Компьютерные шрифты существуют в различных технических форматах. Формат TrueType, разработанный компанией Apple в конце восьмидесятых годов, получил широкое распространение благодаря поддержке со стороны операционной системы Windows. Формат OpenType, созданный совместными усилиями Microsoft и Adobe, расширил возможности шрифтов за счёт поддержки дополнительных типографических функций и большего количества глифов. Веб-шрифты в форматах WOFF и WOFF2 оптимизированы для передачи по сети и используются на веб-страницах.

Отображение шрифта в цифровой среде требует постоянного пересчёта. При изменении размера кегля, ширины текстового блока или масштаба страницы программа заново вычисляет положение каждого символа, переносы слов, межстрочные и межбуквенные интервалы. Этот процесс происходит в текстовых редакторах, издательских системах и веб-браузерах. Чтобы сделать пересёт гибче, существуют вариативные

шрифты (*variable fonts*). Они расширяют возможности динамической адаптации, позволяя плавно изменять насыщенность, ширину и другие параметры начертания без загрузки отдельных файлов.

Электронная типография предполагает наличие парка рабочих станций, на которых выполняются задачи вёрстки, дизайна и допечатной подготовки. Рабочая станция представляет собой персональный компьютер с определёнными аппаратными характеристиками: процессором, объёмом оперативной памяти, типом и ёмкостью накопителя, параметрами графического адаптера. На каждой станции установлена операционная система — Windows, macOS или Linux — определённой версии. Сочетание аппаратных характеристик и операционной системы определяет, какие форматы шрифтов поддерживаются и насколько корректно они отображаются.

Шрифты в электронной типографии — лицензируемые продукты. Лицензия определяет допустимые способы использования: некоторые шрифты распространяются бесплатно для любых целей, другие требуют приобретения коммерческой лицензии, третьи доступны по подписке. Лицензионные условия могут ограничивать количество рабочих станций, на которых разрешена установка шрифта, тираж печатной продукции или число просмотров веб-страниц. Без учёта лицензий типография рискует нарушить авторские права.

В электронной типографии работают несколько категорий специалистов. Системный администратор отвечает за техническое состояние парка рабочих станций, установку операционных систем, настройку программного обеспечения и распределение шрифтовых ресурсов между станциями. Дизайнер осуществляет подбор шрифтов для проектов, формирует запросы на приобретение новых гарнитур и использует установленные шрифты в своей работе. Менеджер по закупкам ведёт учёт лицензий, оформляет приобретение новых шрифтов, контролирует сроки действия подписок и соответствие количества установок лицензионным ограничениям.

Процесс пополнения шрифтовой коллекции начинается с выявления потребности. Дизайнер, работая над проектом, может обнаружить необходимость в гарнитуре, отсутствующей в текущей коллекции типографии. Он формирует запрос, указывая название шрифта, требуемые начертания и обоснование выбора. Менеджер по закупкам проверяет наличие шрифта у издательств, сравнивает условия лицензирования и стоимость. После согласования бюджета оформляется приобретение лицензии. Системный администратор получает файлы шрифта и выполняет установку на рабочие станции, для которых предназначена лицензия.

Допечатная подготовка и печать требуют строгого соответствия шрифтов на всех этапах производства. Верстальщик создаёт макет на своей рабочей станции, используя определённые гарнитуры и начертания. За-

тем файл передаётся на допечатную подготовку, затем на печать. На каждом этапе программа обращается к шрифтам, установленным локально. Если на одной из станций нужный шрифт отсутствует, система подставит замену — похожий шрифт с другими метриками. Это приведёт к смещению строк, изменению интервалов, нарушению переносов и выравнивания. Ошибка может остаться незамеченной до печати тиража, а исправление потребует перевёрстки и дополнительных затрат. Централизованный учёт шрифтов позволяет контролировать идентичность коллекций на всех станциях и предотвращать подобные ситуации ещё на этапе передачи макета.

Процесс инвентаризации шрифтовых ресурсов направлен на поддержание актуальности сведений о коллекции. Системный администратор периодически проводит сверку фактически установленных шрифтов на рабочих станциях с данными учётной системы. Выявляются расхождения: шрифты, установленные без учёта, удалённые шрифты, изменения в конфигурации станций. По результатам инвентаризации учётные данные корректируются. Менеджер по закупкам проверяет соответствие количества установок лицензионным ограничениям и при необходимости приобретает дополнительные лицензии или деактивирует избыточные установки.

Таким образом, учёт компьютерных шрифтов в электронной типографии представляет собой комплексную задачу, охватывающую систематизацию сведений о шрифтовых гарнитурах и их характеристиках, регистрацию парка рабочих станций с их аппаратно-программными конфигурациями, отслеживание лицензионных условий и сроков их действия, а также фиксацию связей между конкретными шрифтами и станциями, на которых они установлены. Организация такого учёта обеспечивает соблюдение лицензионных требований, оперативный поиск необходимых шрифтов и рациональное использование типографических ресурсов.

1.2 Цели функционирования базы данных

1. Хранение сведений о шрифтовых гарнитурах, их начертаниях и характеристиках.
2. Хранение информации о рабочих станциях и их аппаратно-программных конфигурациях.
3. Хранение информации о лицензиях на шрифты, условиях использования и сроках действия.
4. Хранение информации об установленных шрифтах на каждой рабочей станции.

1.3 Польза от внедрения базы данных

Польза от внедрения базы данных напрямую следует из поставленных целей. Единый каталог гарнитур позволяет быстро находить нужный шрифт и проверять его характеристики. Реестр рабочих станций с информацией об установленных шрифтах даёт возможность убедиться, что на всех станциях установлены одни и те же шрифты. Учёт лицензий предотвращает нарушение авторских прав и помогает планировать бюджет на продление подписок. Фиксация связей между шрифтами и станциями упрощает выявление расхождений. В результате типография получает инструмент, обеспечивающий согласованность шрифтовых коллекций, соблюдение лицензионных требований и прозрачность использования типографических ресурсов.

1.4 Описание сущностей

1. Гарнитура — это семейство начертаний шрифта, объединённых общим стилем и предназначенных для совместного использования в типографических работах.

Атрибуты: название, год создания, описание, число начертаний.

2. Начертание — это конкретный вариант гарнитуры, определяемый насыщенностью штрихов и наклоном знаков, хранящийся в отдельном файле шрифта.

Атрибуты: название, насыщенность, наклон, имя файла, размер файла.

3. Категория — это классификационная группа, объединяющая шрифты по конструктивным особенностям строения знаков (например, по наличию засечек или ширине символов).

Атрибуты: название, описание, иконка, число гарнитур.

4. Формат — это технический стандарт хранения шрифтовых данных, определяющий структуру файла, способ описания знаков и совместимость с платформами.

Атрибуты: название, расширение, год появления, описание, поддержка веба.

5. Язык — это система письма с определённым алфавитом и направлением чтения, поддержка которой обеспечивается наличием соответствующих знаков в шрифте *начертании шрифта*.

Атрибуты: название, код ISO, направление письма, тип письменности.

6. Алфавит — это совокупность глифов в конкретном начертании шрифта, обеспечивающая отображение текста на определённом языке.

Атрибуты: число начертаний в системе, число полностью поддерживаемых гарнитур, дата проверки, статус.

7. Лицензия — это документ, устанавливающий права и ограничения на использование шрифта, включая количество рабочих станций и срок действия; на каждую гарнитуру распространяются отдельные лицензии.

Атрибуты: тип, срок действия, максимальное число станций, стоимость, дата приобретения.

8. Издательство — это организация, занимающаяся разработкой, производством и распространением шрифтов, а также предоставлением лицензий на их использование.

Атрибуты: название, страна, сайт, год основания, контактный email.

9. Рабочая станция — это персональный компьютер в составе парка типографии, используемый для выполнения задач вёрстки, дизайна и допечатной подготовки.

Атрибуты: инвентарный номер, процессор, объём памяти, тип накопителя, операционная система.

10. Операционная система — это комплекс программ, управляющий аппаратными ресурсами рабочей станции и обеспечивающий среду для работы прикладного программного обеспечения.

Атрибуты: название, версия, разрядность, год выпуска.

11. Использование — это факт размещения конкретного начертания шрифта на рабочей станции, фиксирующий связь между шрифтовым ресурсом и оборудованием.

Атрибуты: дата установки, дата удаления, путь к файлу, статус, дата последнего использования.

1.5 Описание ролей и субъектов

Системный администратор обслуживает парк рабочих станций. Он устанавливает операционные системы, настраивает программное обеспечение и распределяет шрифты между станциями.

Дизайнер использует шрифты в своей работе. Он подбирает гарнитуры для проектов и формирует запросы на приобретение новых шрифтов.

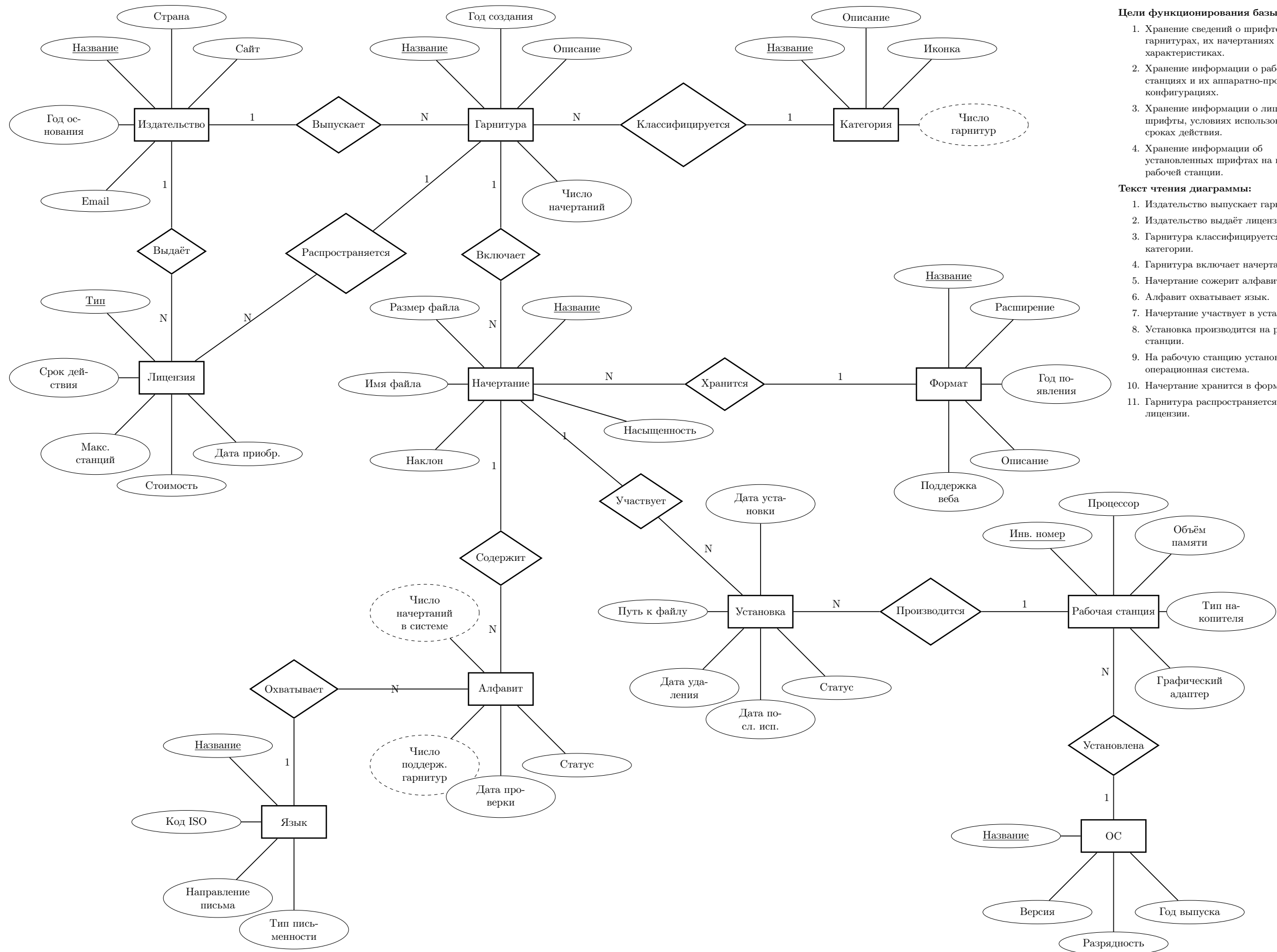
Менеджер по закупкам ведёт учёт лицензий. Он оформляет приобретение шрифтов, контролирует сроки действия подписок и соответствие количества установок лицензионным ограничениям.

Издательство выпускает шрифты и предоставляет лицензии на их использование. Типография приобретает лицензии у издательств и устанавливает шрифты на рабочие станции.

1.6 ER-диаграмма

На рисунке 1 (стр. 10) представлена ER-диаграмма предметной области. Перечень связей:

1. Издательство выпускает гарнитуру.
2. Издательство выдаёт лицензию.
3. Гарнитура классифицируется по категории.
4. Гарнитура включает начертание.
5. Начертание сожержит алфавит.
6. Алфавит охватывает язык.
7. Начертание участвует в установке.
8. Установка производится на рабочей станции.
9. На рабочую станцию установлена операционная система.
10. Начертание хранится в формате.
11. Гарнитура распространяется по лицензии.



Цели функционирования базы данных:

1. Хранение сведений о шрифтовых гарнитурах, их начертаниях и характеристиках.
2. Хранение информации о рабочих станциях и их аппаратно-программных конфигурациях.
3. Хранение информации о лицензиях на шрифты, условиях использования и сроках действия.
4. Хранение информации об установленных шрифтах на каждой рабочей станции.

Текст чтения диаграммы:

1. Издательство выпускает гарнитуру.
2. Издательство выдаёт лицензию.
3. Гарнитура классифицируется по категории.
4. Гарнитура включает начертание.
5. Начертание содержит алфавит.
6. Алфавит охватывает язык.
7. Начертание участвует в установке.
8. Установка производится на рабочей станции.
9. На рабочую станцию установлена операционная система.
10. Начертание хранится в формате.
11. Гарнитура распространяется по лицензии.

Рис. 1: ER-диаграмма предметной области «Учёт компьютерных шрифтов в электронной типографии»

1.7 Схема объектов

Схема объектов представлена на рисунке 2.

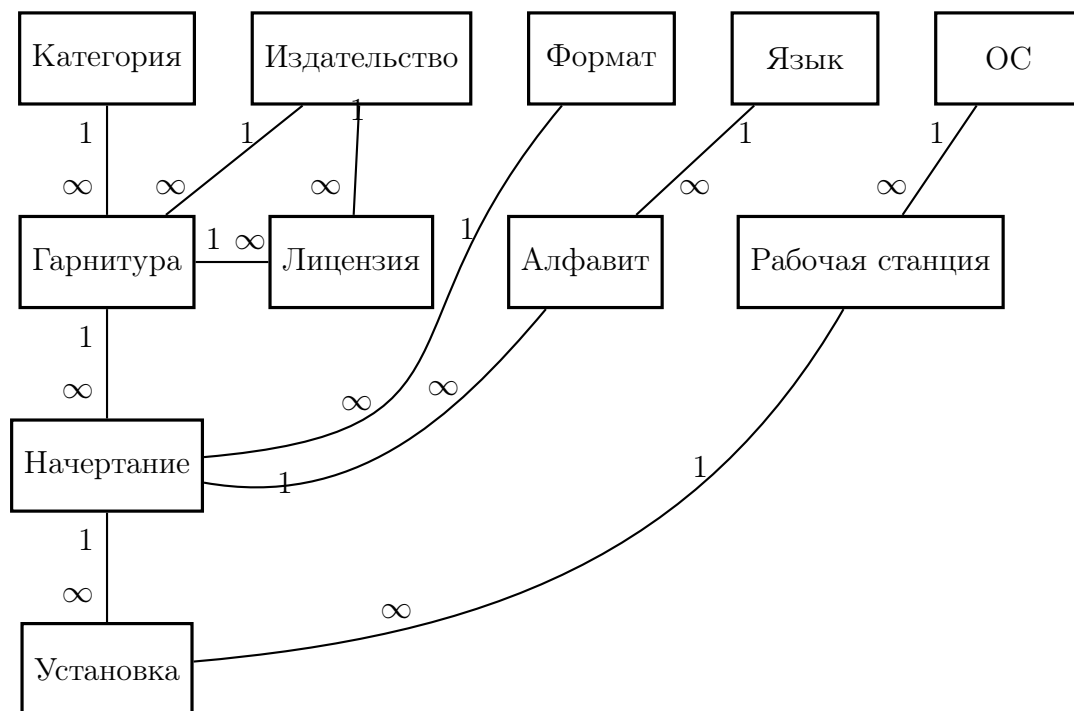


Рис. 2: Схема объектов предметной области «Учёт компьютерных шрифтов в электронной типографии»

2 Проектирование

Схема базы данных приведена на рисунках 3 и 4 на русском и английском языках соответственно.

2.1 Лингвистическое описание схемы базы данных

1. *Издательство* (*id_Издательство* INT, *Название* VARCHAR(100), *Страна* VARCHAR(60), *Сайт* TEXT, *Год_основания* SMALLINT, *Email* VARCHAR(254)).
2. *Гарнитура* (*id_Гарнитура* INT, *Название* VARCHAR(100), *Год_создания* SMALLINT, *Описание* TEXT, *Число_начертаний* SMALLINT, *id_Издательство* INT, *id_Категория* INT).
3. *Категория* (*id_Категория* INT, *Название* VARCHAR(100), *Описание* TEXT, *Иконка* TEXT, *Число_гарнитур* INT).
4. *Лицензия* (*id_Лицензия* INT, *Тип* VARCHAR(30), *Срок_действия* DATE, *Максимальное_число_станций* SMALLINT, *Стоимость* DECIMAL(10,2), *Дата_приобретения* DATE, *id_Издательство* INT, *id_Гарнитура* INT).
5. *Формат* (*id_Формат* INT, *Название* VARCHAR(100), *Расширение* VARCHAR(10), *Год_появления* SMALLINT, *Описание* TEXT, *Поддержка_веба* BOOLEAN).
6. *Начертание* (*id_Начертание* INT, *Название* VARCHAR(100), *Насыщенность* VARCHAR(30), *Наклон* VARCHAR(30), *Размер_файла* INT, *id_Гарнитура* INT, *id_Формат* INT).
7. *Язык* (*id_Язык* INT, *Название* VARCHAR(60), *Код_ISO* CHAR(3), *Направление_письма* VARCHAR(30), *Тип_письменности* VARCHAR(30)).
8. *Алфавит* (*id_Алфавит* INT, *Число_начертаний_в_системе* INT, *Число_поддерживаемых_гарнитур* INT, *Дата_проверки* DATE, *Статус* VARCHAR(30), *id_Начертание* INT, *id_Язык* INT).
9. *Рабочая_станция* (*id_Рабочая_станция* INT, *Инвентарный_номер* VARCHAR(20), *Процессор* VARCHAR(80), *Объём_памяти* INT, *Тип_накопителя* VARCHAR(30), *id_ОС* INT).
10. *ОС* (*id_ОС* INT, *Название* VARCHAR(50), *Версия* VARCHAR(30), *Разрядность* SMALLINT, *Год_выпуска* SMALLINT).
11. *Установка* (*id_Установка* INT, *Дата_установки* DATE, *Дата_удаления* DATE, *Путь_к_файлу* TEXT, *Статус* VARCHAR(30), *Дата_последнего_использования* DATE, *id_Начертание* INT, *id_Рабочая_станция* INT).

2.2 Описание таблиц базы данных

В таблицах 1–11 представлены атрибуты проектируемой базы данных. В полях таблиц перечислены: ключ (*тип ключа «PK» — Primary Key, «FK» — Foreign Key*), названия атрибутов на русском и на английском языках, тип атрибута и ссылка на другую таблицу, если таковая имеется.

Таблица 1: Издательство — Publisher

Издательство — Publisher						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_Publisher	id_Издательство	INT	NOT NULL	—
2	—	Name	Название	VARCHAR(100)	NOT NULL	—
3	—	Country	Страна	VARCHAR(60)	NOT NULL	—
4	—	Website	Сайт	TEXT	NULL	—
5	—	Foundation_year	Год_основания	SMALLINT	NOT NULL	—
6	—	Email	Email	VARCHAR(254)	NULL	—

В таблице 1 для URL-адресов выбран тип `TEXT`, так как считаем, что в окне браузера можно ввести URL-адрес любой длины. Тип `VARCHAR(254)` выбран для email-адресов. Их максимальная длина составляет 254 символа согласно RFC 5321¹. `VARCHAR(100)` — для имён гарнитур и начертаний² (аналогично в таблицах 2, 3, 5 и 6) тогда как `VARCHAR(60)` ограничивает названия стран в соответствии с максимальной длиной официальных наименований. Для стоимости лицензии применяется `DECIMAL(10,2)`, обеспечивающий точные денежные вычисления без ошибок округления. Код языка хранится в `CHAR(3)`, так как коды ISO 639³ имеют фиксированную длину в два или три символа.

¹RFC 5321, Section 4.5.3.1 — <https://datatracker.ietf.org/doc/html/rfc5321#section-4.5.3.1>

²Например, самое длинное название начертания, которое удалось найти «Noto Sans Devanagari SemiCondensed ExtraLight Italic», — 52 символа.

³<https://www.iso.org/iso-639-language-code>

Таблица 2: Гарнитура — Font_family

<i>Гарнитура — Font_family</i>						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_Font_family	id_Гарнитура	INT	NOT NULL	—
2	—	Name	Название	VARCHAR(100)	NOT NULL	—
3	—	Year_created	Год_создания	SMALLINT	NULL	—
4	—	Description	Описание	TEXT	NULL	—
5	—	Typeface_count	Число_начертаний	SMALLINT	NOT NULL	—
6	FK	id_Publisher	id_Издательство	INT	NOT NULL	Publisher.id_Publisher
7	FK	id_Category	id_Категория	INT	NOT NULL	Category.id_Category

Таблица 3: Категория — Category

<i>Категория — Category</i>						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_Category	id_Категория	INT	NOT NULL	—
2	—	Name	Название	VARCHAR(100)	NOT NULL	—
3	—	Description	Описание	TEXT	NULL	—
4	—	Icon	Иконка	TEXT	NULL	—
5	—	Font_family_count	Число_гарнитур	INT	NOT NULL	—

В таблицах 3 и 11 имя директории имеет тип TEXT, т.к. может быть любой длины в рамках операционной системы.

Таблица 4: Лицензия — License

<i>Лицензия — License</i>						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_License	id_Лицензия	INT	NOT NULL	—
2	—	Type	Тип	VARCHAR(30)	NOT NULL	—
3	—	Expiration_date	Срок_действия	DATE	NULL	—
4	—	Max_stations	Макс_число_станций	SMALLINT	NULL	—
5	—	Cost	Стоимость	DECIMAL(10,2)	NOT NULL	—
6	—	Purchase_date	Дата_приобретения	DATE	NOT NULL	—
7	FK	id_Publisher	id_Издательство	INT	NOT NULL	Publisher.id_Publisher
8	FK	id_Font_family	id_Гарнитура	INT	NOT NULL	Font_family.id_Font_family

В таблице 4 тип лицензии хранится в `VARCHAR(30)`, что достаточно для значений вроде «OFL», «Commercial», «Subscription» или «Trial». Поле `Срок_действия` допускает `NULL`, поскольку бессрочные лицензии не имеют даты окончания. Аналогично `Макс_число_станций` допускает `NULL` для лицензий без ограничения на количество установок.

Таблица 5: Формат — Format

<i>Формат — Format</i>						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_Format	id_Формат	INT	NOT NULL	—
2	—	Name	Название	VARCHAR(100)	NOT NULL	—
3	—	Extension	Расширение	VARCHAR(10)	NOT NULL	—
4	—	Year_released	Год_появления	SMALLINT	NULL	—
5	—	Description	Описание	TEXT	NULL	—
6	—	Web_support	Поддержка_веба	BOOLEAN	NOT NULL	—

В таблице 5 расширение файла хранится в `VARCHAR(10)`, что покрывает существующие форматы шрифтов: «.ttf», «.otf», «.woff2».

Таблица 6: Начертание — Typeface

<i>Начертание — Typeface</i>						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_Typeface	id_Начертание	INT	NOT NULL	—
2	—	Name	Название	VARCHAR(100)	NOT NULL	—
3	—	Weight	Насыщенность	VARCHAR(30)	NOT NULL	—
4	—	Slope	Наклон	VARCHAR(30)	NOT NULL	—
5	—	File_size	Размер_файла	INT	NOT NULL	—
6	FK	id_Font_family	id_Гарнитура	INT	NOT NULL	Font_family.id_Font_family
7	FK	id_Format	id_Формат	INT	NOT NULL	Format.id_Format

В таблице 6 насыщенность и наклон хранятся в `VARCHAR(30)`, что достаточно для стандартных значений: «Thin», «ExtraLight», «Regular», «Bold», «ExtraBold» для насыщенности и «Upright», «Italic», «Oblique» для наклона. Размер файла хранится в `INT` и измеряется в байтах — максимальное значение около 2 ГБ, что с запасом покрывает любые файлы шрифтов.

Таблица 7: Язык — Language

<i>Язык — Language</i>						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_Language	id_Язык	INT	NOT NULL	—
2	—	Name	Название	VARCHAR(60)	NOT NULL	—
3	—	ISO_code	Код_ISO	CHAR(3)	NOT NULL	—
4	—	Writing_direction	Направление_письма	VARCHAR(30)	NOT NULL	—
5	—	Script_type	Тип_письменности	VARCHAR(30)	NOT NULL	—

В таблице 7 название языка хранится в `VARCHAR(60)`, что покрывает самые длинные наименования в реестре ISO 639-3⁴, например «Min Nan Chinese» или «Ancient Greek». Направление письма хранится в `VARCHAR(30)` для значений «left-to-right», «right-to-left» или «top-to-bottom». Тип письменности — «alphabetic», «syllabic», «logographic» — также укладывается в `VARCHAR(30)`. Аналогично со статусом в таблице 8 — также хранится в `VARCHAR(30)` и принимает стандартные значения «full», «partial»

⁴ISO 639-3 Code Tables — https://iso639-3.sil.org/code_tables/639/data

или «unsupported». Или в таблице 11 поле Статус принимает значения «active», «disabled» или «removed» — тоже используется тип `VARCHAR(30)`.

Таблица 8: Алфавит — Alphabet

<i>Алфавит — Alphabet</i>						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_Alphabet	id_Алфавит	INT	NOT NULL	—
2	—	Typeface_count_in_system	Число_начертаний_в_системе	INT	NOT NULL	—
3	—	Supported_family_count	Число_поддерживаемых_гарнитур	INT	NOT NULL	—
4	—	Verification_date	Дата_проверки	DATE	NULL	—
5	—	Status	Статус	VARCHAR(30)	NOT NULL	—
6	FK	id_Typeface	id_Начертание	INT	NOT NULL	Typeface.id_Typeface
7	FK	id_Language	id_Язык	INT	NOT NULL	Language.id_Language

Таблица 9: Рабочая станция — Workstation

<i>Рабочая станция — Workstation</i>						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_Workstation	id_Рабочая_станция	INT	NOT NULL	—
2	—	Inventory_number	Инвентарный_номер	VARCHAR(20)	NOT NULL	—
3	—	Processor	Процессор	VARCHAR(80)	NOT NULL	—
4	—	Memory_size	Объём_памяти	INT	NOT NULL	—
5	—	Storage_type	Тип_накопителя	VARCHAR(30)	NOT NULL	—
6	FK	id_OS	id_ОС	INT	NOT NULL	OS.id_OS

В таблице 9 название процессора хранится в `VARCHAR(80)`: согласно спецификации Intel⁵, название процессора занимает до 48 символов, а запас до 80 учитывает возможные дополнения при ручном вводе. Объём памяти хранится в `INT` и измеряется в мегабайтах. Тип накопителя — `VARCHAR(30)` для значений «SSD», «HDD» или «NVMe SSD». Инвентарный номер — `VARCHAR(20)`, поскольку инвентарные номера обычно содержат буквенно-цифровые коды фиксированной длины, считаем, что в нашей типографии максимальная длина кода — 20 символов.

⁵Intel® Processor Identification and the CPUID Instruction, Application Note 485, Section 3.2.3 — <https://datasheets.chipdb.org/Intel/x86/CPUID/24161831.pdf>

Таблица 10: Операционная система — OS

<i>Операционная система — OS</i>						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_OS	id_ОС	INT	NOT NULL	—
2	—	Name	Название	VARCHAR(50)	NOT NULL	—
3	—	Version	Версия	VARCHAR(30)	NOT NULL	—
4	—	Bitness	Разрядность	SMALLINT	NOT NULL	—
5	—	Release_year	Год_выпуска	SMALLINT	NOT NULL	—

В таблице 10 название операционной системы хранится в `VARCHAR(50)`: согласно реестру IANA⁶, официальное имя ОС может содержать до 40 символов, а запас до 50 учитывает неофициальные наименования. Версия — в `VARCHAR(30)`, поскольку версии могут содержать буквенно-цифровые обозначения, например «24H2», «15.4.1» или «24.04 LTS». Разрядность хранится в `SMALLINT` и принимает значения 32 или 64.

Таблица 11: Установка — Installation

<i>Установка — Installation</i>						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_Installation	id_Установка	INT	NOT NULL	—
2	—	Installation_date	Дата_установки	DATE	NOT NULL	—
3	—	Removal_date	Дата_удаления	DATE	NULL	—
4	—	File_path	Путь_к_файлу	TEXT	NOT NULL	—
5	—	Status	Статус	VARCHAR(30)	NOT NULL	—
6	—	Last_used_date	Дата_последнего_использования	DATE	NULL	—
7	FK	id_Typeface	id_Начертание	INT	NOT NULL	Typeface.id_Typeface
8	FK	id_Workstation	id_Рабочая_станция	INT	NOT NULL	Workstation.id_Workstation

⁶IANA Operating System Names — <https://www.iana.org/assignments/operating-system-names>

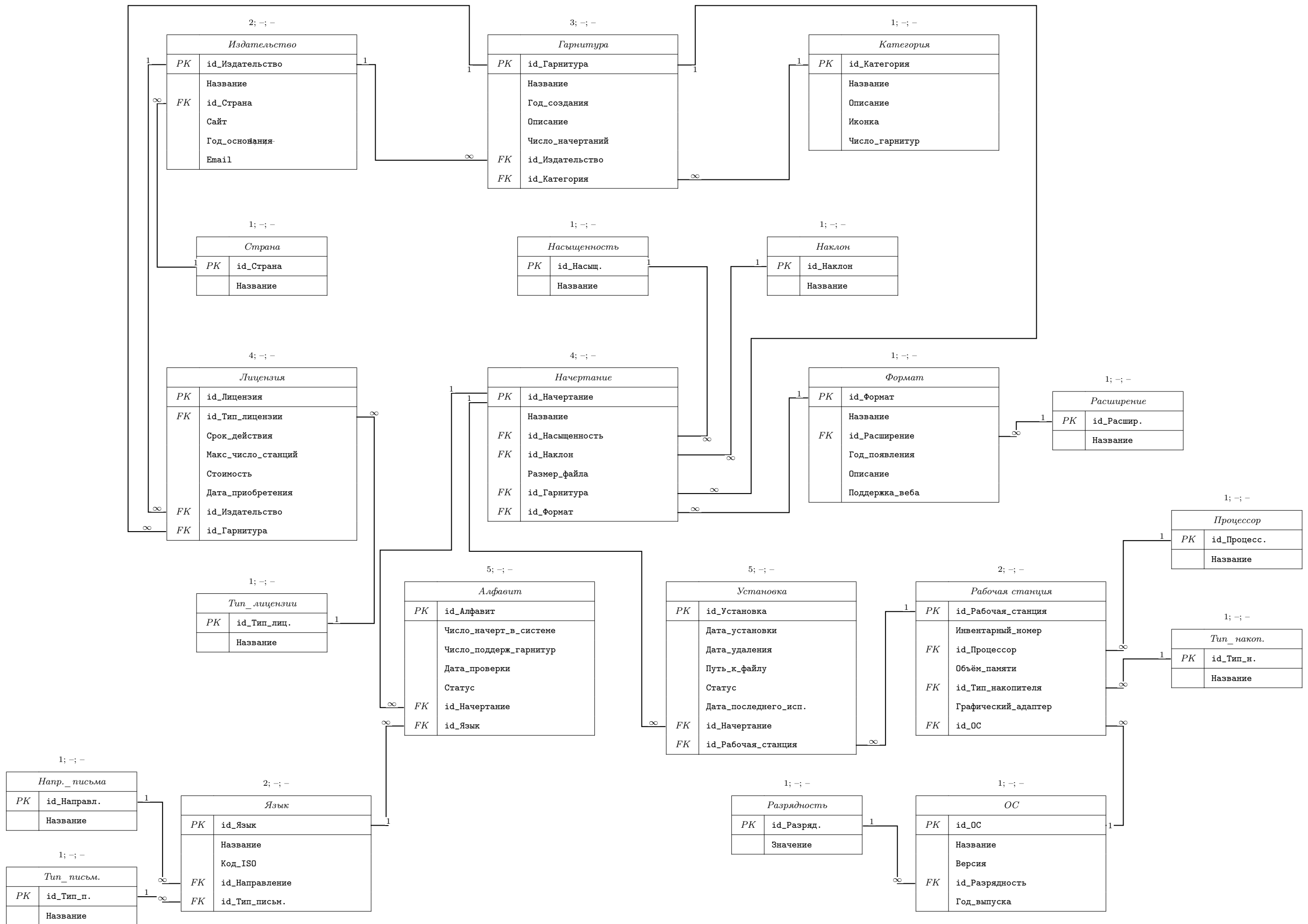


Рис. 3: Схема базы данных на русском языке

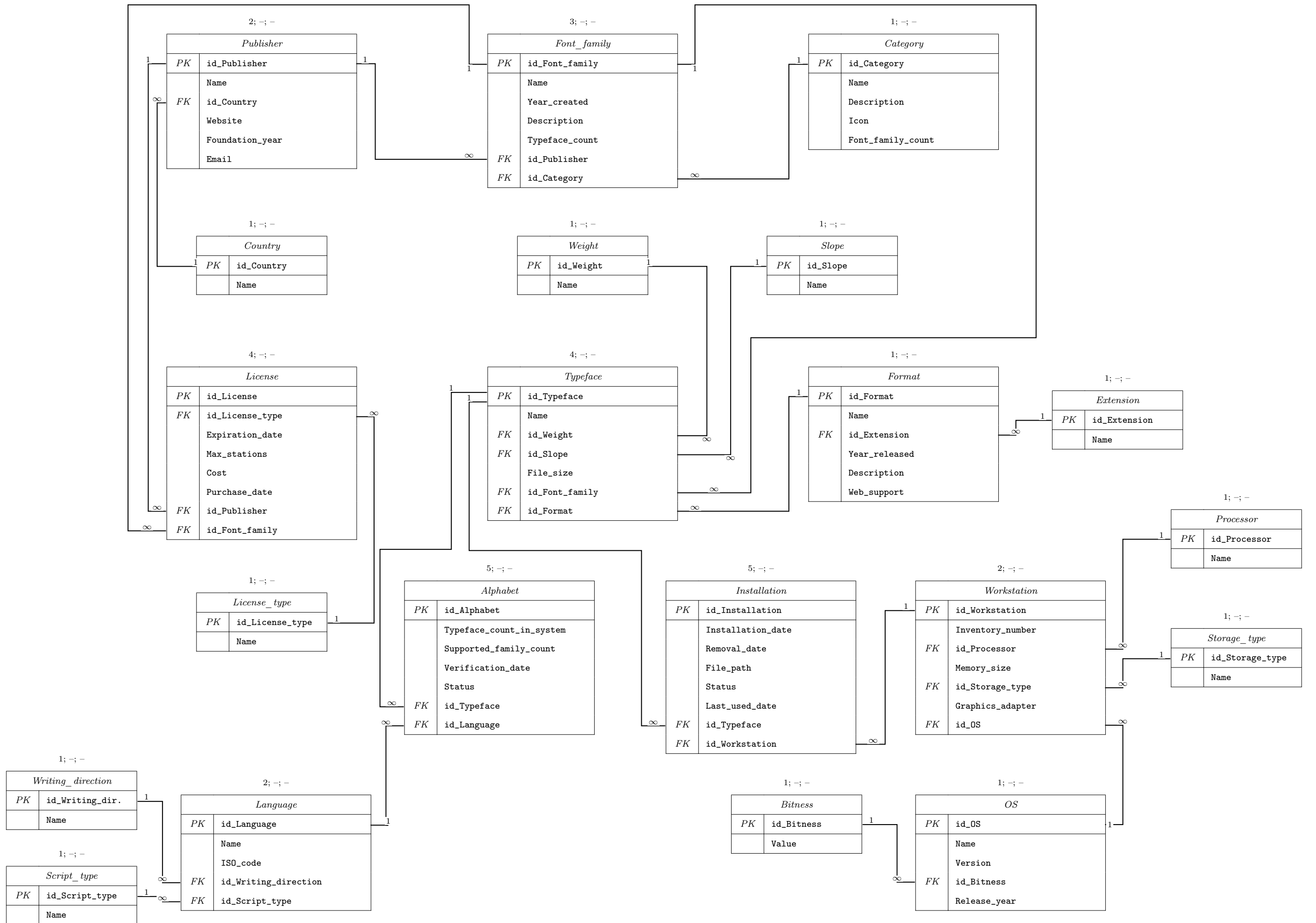


Рис. 4: Схема базы данных на английском языке

2.3 Создание таблиц базы данных и генерация записей

Для создания таблиц базы данных был написан SQL-скрипт для СУБД PostgreSQL ([приложение А](#)). В проекте используется база данных `typography`, запущенная в контейнере `Docker`. Контейнер базы данных создаётся на основе образа PostgreSQL и доступен на локальном порту 5432.

В указанном выше SQL-скрипте последовательно создаются таблицы предметной области: `Category`, `Font_family`, `Format`, `Typeface`, `Language` и `Alphabet`. Порядок создания таблиц выбран с учётом внешних ключей: сначала создаются справочные и независимые таблицы, затем таблицы, которые на них ссылаются.

Структура связей между таблицами, используемыми при программной реализации базы данных, представлена на уменьшенных схемах базы данных на рисунках [5](#) и [6](#).

Для заполнения базы данных тестовыми данными используется программа на TypeScript ([приложение Б](#)). Подключение к базе данных выполняется при помощи библиотеки `pg`. Для генерации правдоподобных значений используется библиотека `@faker-js/faker`. Запуск скриптов выполняется в среде Node.js с использованием `tsx`.

Генерация записей должна выполняться в том же порядке, что и создание зависимостей между таблицами. Сначала заполняются таблицы `Category`, `Format` и `Language`, так как они не зависят от других таблиц. Затем создаются записи в таблице `Font_family`, для каждой из которых выбирается существующая категория. После этого создаются записи в таблице `Typeface`, где для каждого начертания выбираются существующие гарнитура и формат. Последней заполняется таблица `Alphabet`, связывающая существующие языки и начертания.

Количество записей в каждой таблице приведено на схемах базы данных (рисунки [5](#) и [6](#)).

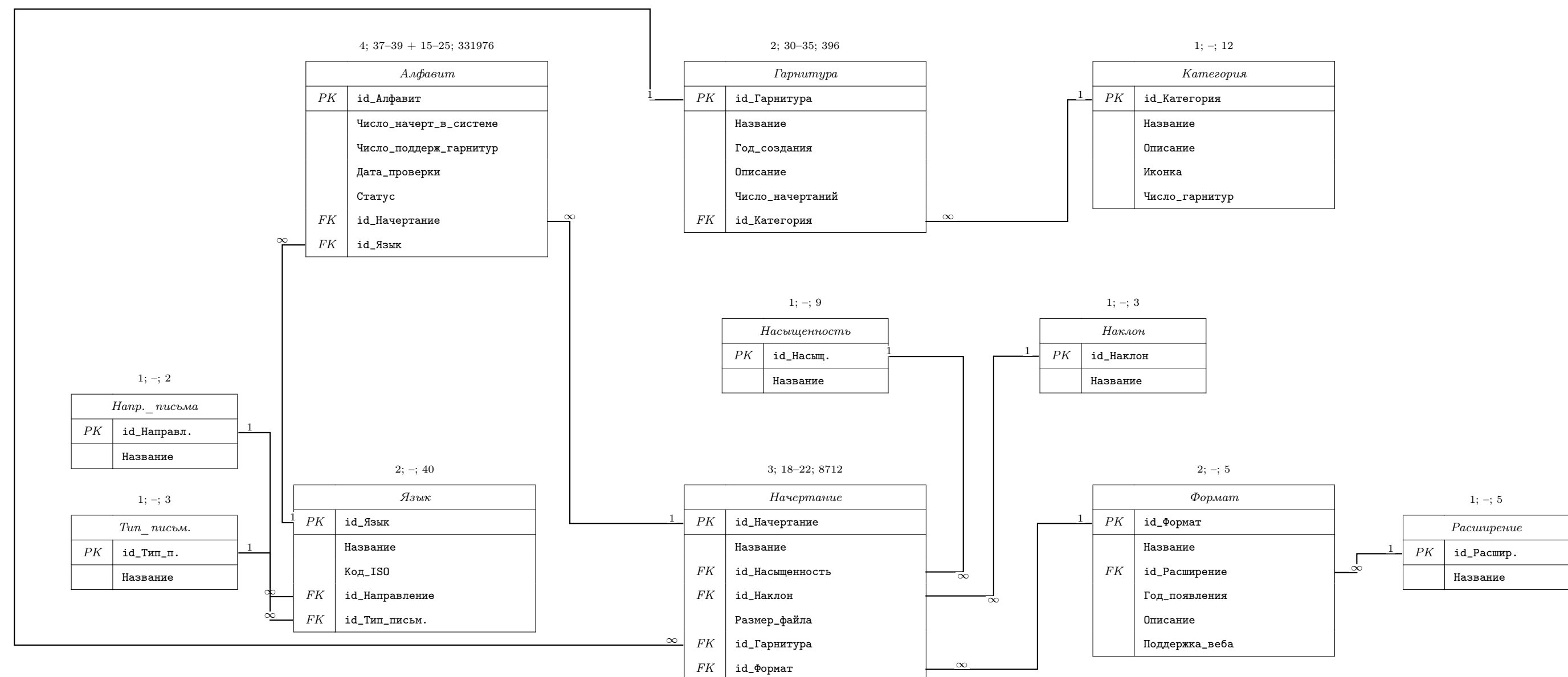


Рис. 5: Программируемая схема базы данных

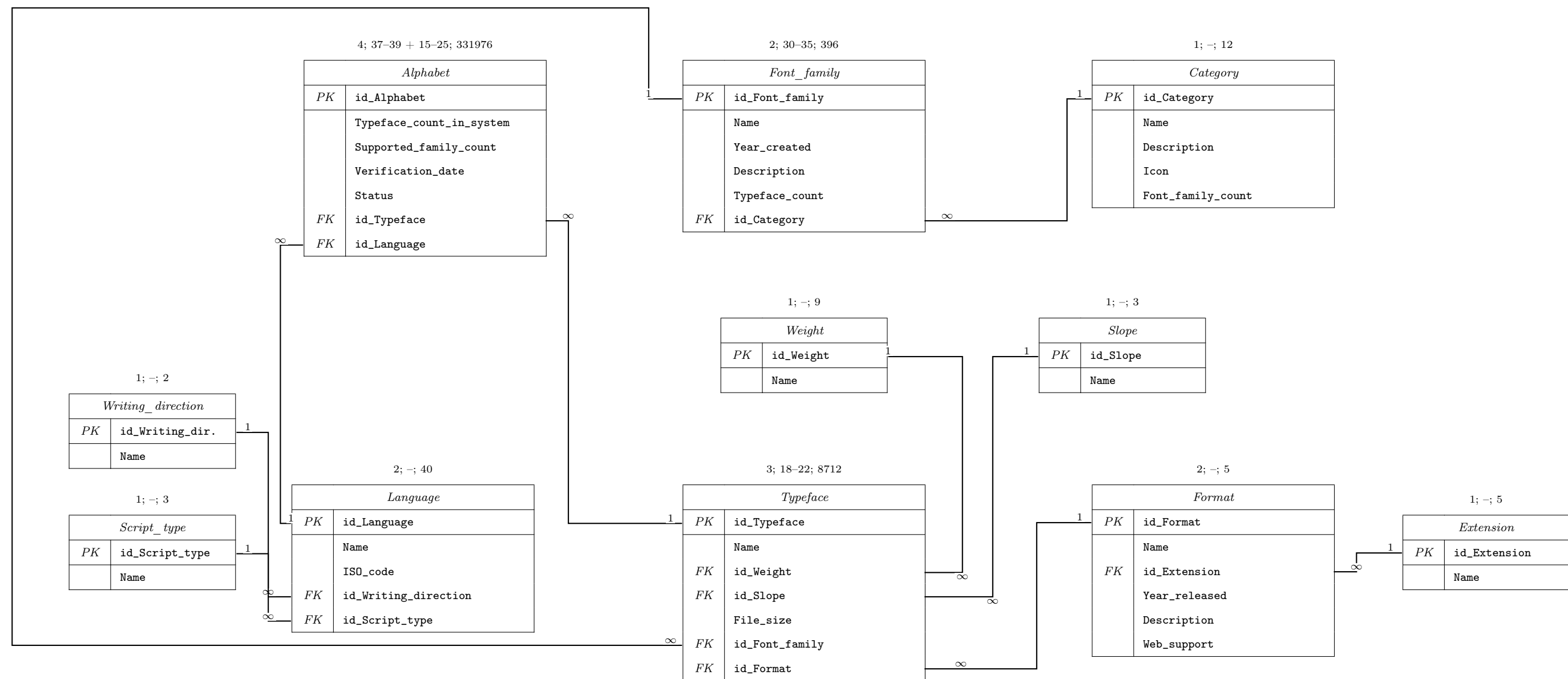


Рис. 6: Программируемая схема базы данных на английском языке

3 Программирование

Приложение А. Исходный код SQL-скрипта для создания таблиц базы данных

```
1 DROP TABLE IF EXISTS Alphabet CASCADE;
2
3 DROP TABLE IF EXISTS Typeface CASCADE;
4
5 DROP TABLE IF EXISTS Font_family CASCADE;
6
7 DROP TABLE IF EXISTS Language CASCADE;
8
9 DROP TABLE IF EXISTS Format CASCADE;
10
11 DROP TABLE IF EXISTS Category CASCADE;
12
13 DROP TABLE IF EXISTS Weight CASCADE;
14
15 DROP TABLE IF EXISTS Slope CASCADE;
16
17 DROP TABLE IF EXISTS Extension CASCADE;
18
19 DROP TABLE IF EXISTS ISO_code CASCADE;
20
21 DROP TABLE IF EXISTS Writing_direction CASCADE;
22
23 DROP TABLE IF EXISTS Script_type CASCADE;
24
25 CREATE TABLE Category (
26     id_Category SERIAL PRIMARY KEY,
27     Name VARCHAR(100) NOT NULL,
28     Description TEXT,
29     Icon TEXT,
30     Font_family_count INT NOT NULL DEFAULT 0
31 );
32
33 CREATE TABLE Weight (
34     id_Weight SERIAL PRIMARY KEY,
35     Name VARCHAR(30) NOT NULL UNIQUE
36 );
37
38 CREATE TABLE Slope (
39     id_Slope SERIAL PRIMARY KEY,
40     Name VARCHAR(30) NOT NULL UNIQUE
41 );
42
43 CREATE TABLE Extension (
44     id_Extension SERIAL PRIMARY KEY,
45     Name VARCHAR(10) NOT NULL UNIQUE
46 );
47
48 CREATE TABLE Writing_direction (
49     id_Writing_direction SERIAL PRIMARY KEY,
50     Name VARCHAR(30) NOT NULL UNIQUE
51 );
52
53 CREATE TABLE Script_type (
54     id_Script_type SERIAL PRIMARY KEY,
55     Name VARCHAR(30) NOT NULL UNIQUE
56 );
57
58 CREATE TABLE Format (
59     id_Format SERIAL PRIMARY KEY,
60     Name VARCHAR(100) NOT NULL,
61     id_Extension INT NOT NULL REFERENCES Extension(id_Extension),
62     Year_released SMALLINT,
63     Description TEXT,
64     Web_support BOOLEAN NOT NULL
65 );
66
67 CREATE TABLE Language (
68     id_Language SERIAL PRIMARY KEY,
69     Name VARCHAR(60) NOT NULL,
70     ISO_code CHAR(3) NOT NULL UNIQUE,
71     id_Writing_direction INT NOT NULL REFERENCES Writing_direction(id_Writing_direction),
72     id_Script_type INT NOT NULL REFERENCES Script_type(id_Script_type)
```

```

73 );
74
75 CREATE TABLE Font_family (
76     id_Font_family SERIAL PRIMARY KEY,
77     Name VARCHAR(100) NOT NULL,
78     Year_created SMALLINT,
79     Description TEXT,
80     Typeface_count SMALLINT NOT NULL DEFAULT 0,
81     id_Category INT NOT NULL REFERENCES Category(id_Category)
82 );
83
84 CREATE TABLE Typeface (
85     id_Typeface SERIAL PRIMARY KEY,
86     Name VARCHAR(100) NOT NULL,
87     id_Weight INT NOT NULL REFERENCES Weight(id_Weight),
88     id_Slope INT NOT NULL REFERENCES Slope(id_Slope),
89     File_size INT NOT NULL,
90     id_Font_family INT NOT NULL REFERENCES Font_family(id_Font_family),
91     id_Format INT NOT NULL REFERENCES Format(id_Format)
92 );
93
94 CREATE TABLE Alphabet (
95     id_Alphabet SERIAL PRIMARY KEY,
96     Typeface_count_in_system INT NOT NULL DEFAULT 0,
97     Supported_family_count INT NOT NULL DEFAULT 0,
98     Verification_date DATE,
99     Status VARCHAR(30) NOT NULL,
100     id_Typeface INT NOT NULL REFERENCES Typeface(id_Typeface),
101     id_Language INT NOT NULL REFERENCES Language(id_Language),
102
103     CONSTRAINT uq_alphabet_typeface_language UNIQUE (id_Typeface, id_Language)
104 );

```

Приложение Б. Программа на TypeScript для заполнения базы данных тестовыми данными

```
1
2 // db.ts
3 import pg from "pg";
4
5 export const pool = new pg.Pool({
6   host: "localhost",
7   port: 5432,
8   database: "typography",
9   user: "ilyasemenov",
10  password: "type",
11 });
12
13
14 // init.ts
15 import { readFileSync } from "fs";
16 import { pool } from "./db.js";
17
18 async function init() {
19   const sql = readFileSync("init.sql", "utf-8");
20   await pool.query(sql);
21   console.log("Tables created successfully");
22   await pool.end();
23 }
24
25 init().catch(console.error);
26
27
28 // constants.ts
29 import { faker } from "@faker-js/faker";
30
31 export const BASE_COUNTS = {
32   categories: 12,
33   formats: 5,
34   languages: 40,
35 } as const;
36
37 export const COEFFICIENT_LIMITS = {
38   fontFamiliesPerCategory: { min: 30, max: 35 },
39   typefacesPerFamily: { min: 18, max: 22 },
40   alphabetsPerTypeface: { min: 37, max: 39 },
41   alphabetsPerLanguage: { min: 15, max: 25 },
42 } as const;
43
44 export const COEFFICIENTS = {
45   fontFamiliesPerCategory: faker.number.int(
46     COEFFICIENT_LIMITS.fontFamiliesPerCategory,
47   ),
48
49   typefacesPerFamily: faker.number.int(COEFFICIENT_LIMITS.typefacesPerFamily),
50
51   alphabetsPerTypeface: faker.number.int(
52     COEFFICIENT_LIMITS.alphabetsPerTypeface,
53   ),
54
55   alphabetsPerLanguage: faker.number.int(
56     COEFFICIENT_LIMITS.alphabetsPerLanguage,
57   ),
58 } as const;
59
60 export const COUNTS = {
61   categories: BASE_COUNTS.categories,
62   formats: BASE_COUNTS.formats,
63   languages: BASE_COUNTS.languages,
64
65   fontFamilies: BASE_COUNTS.categories * COEFFICIENTS.fontFamiliesPerCategory,
66
67   typefaces:
68     BASE_COUNTS.categories *
69     COEFFICIENTS.fontFamiliesPerCategory *
70     COEFFICIENTS.typefacesPerFamily,
71
72   alphabets:
```

```

73     BASE_COUNTS.languages * COEFFICIENTS.alphabetsPerLanguage +
74     BASE_COUNTS.categories *
75     COEFFICIENTS.fontFamiliesPerCategory *
76     COEFFICIENTS.typefacesPerFamily *
77     COEFFICIENTS.alphabetsPerTypeface,
78 } as const;
79 export const categoryNames = [
80     "Serif",
81     "Sans-serif",
82     "Monospace",
83     "Display",
84     "Handwriting",
85     "Slab-serif",
86     "Geometric",
87     "Humanist",
88     "Grotesque",
89     "Neo-grotesque",
90     "Transitional",
91     "Didone",
92 ];
93
94 export const categoryDescs: Record<string, string> = {
95     Serif: "Typefaces with decorative strokes at the ends of letterforms",
96     "Sans-serif": "Typefaces without decorative strokes",
97     Monospace: "Typefaces where each character occupies the same width",
98     Display: "Typefaces designed for headlines and large sizes",
99     Handwriting: "Typefaces imitating handwritten text",
100    "Slab-serif": "Typefaces with thick rectangular serifs",
101    Geometric: "Sans-serif typefaces based on geometric shapes",
102    Humanist: "Typefaces with organic strokes inspired by calligraphy",
103    Grotesque: "Early sans-serif typefaces with irregular proportions",
104    "Neo-grotesque": "Refined sans-serif typefaces with uniform strokes",
105    Transitional: "Serif typefaces bridging old-style and modern designs",
106    Didone: "High-contrast serif typefaces with thin hairlines",
107 };
108
109 export const formats = [
110     {
111         name: "TrueType",
112         ext: ".ttf",
113         year: 1991,
114         desc: "Scalable font format developed by Apple",
115         web: false,
116     },
117     {
118         name: "OpenType",
119         ext: ".otf",
120         year: 1996,
121         desc: "Scalable font format by Microsoft and Adobe",
122         web: false,
123     },
124     {
125         name: "WOFF",
126         ext: ".woff",
127         year: 2010,
128         desc: "Web Open Font Format",
129         web: true,
130     },
131     {
132         name: "WOFF2",
133         ext: ".woff2",
134         year: 2014,
135         desc: "Compressed Web Open Font Format",
136         web: true,
137     },
138     {
139         name: "TrueType Collection",
140         ext: ".ttc",
141         year: 1994,
142         desc: "Multiple TrueType fonts in a single file",
143         web: false,
144     },
145 ];
146
147 export const languages = [
148     {
149         name: "English",
150         iso: "eng",

```

```

151     writingDirection: "left-to-right",
152     scriptType: "alphabetic",
153 },
154 {
155     name: "Russian",
156     iso: "rus",
157     writingDirection: "left-to-right",
158     scriptType: "alphabetic",
159 },
160 {
161     name: "German",
162     iso: "deu",
163     writingDirection: "left-to-right",
164     scriptType: "alphabetic",
165 },
166 {
167     name: "French",
168     iso: "fra",
169     writingDirection: "left-to-right",
170     scriptType: "alphabetic",
171 },
172 {
173     name: "Spanish",
174     iso: "spa",
175     writingDirection: "left-to-right",
176     scriptType: "alphabetic",
177 },
178 {
179     name: "Italian",
180     iso: "ita",
181     writingDirection: "left-to-right",
182     scriptType: "alphabetic",
183 },
184 {
185     name: "Portuguese",
186     iso: "por",
187     writingDirection: "left-to-right",
188     scriptType: "alphabetic",
189 },
190 {
191     name: "Dutch",
192     iso: "nld",
193     writingDirection: "left-to-right",
194     scriptType: "alphabetic",
195 },
196 {
197     name: "Polish",
198     iso: "pol",
199     writingDirection: "left-to-right",
200     scriptType: "alphabetic",
201 },
202 {
203     name: "Czech",
204     iso: "ces",
205     writingDirection: "left-to-right",
206     scriptType: "alphabetic",
207 },
208 {
209     name: "Slovak",
210     iso: "slk",
211     writingDirection: "left-to-right",
212     scriptType: "alphabetic",
213 },
214 {
215     name: "Ukrainian",
216     iso: "ukr",
217     writingDirection: "left-to-right",
218     scriptType: "alphabetic",
219 },
220 {
221     name: "Belarusian",
222     iso: "bel",
223     writingDirection: "left-to-right",
224     scriptType: "alphabetic",
225 },
226 {
227     name: "Serbian",
228     iso: "srp",

```

```

229     writingDirection: "left-to-right",
230     scriptType: "alphabetic",
231 },
232 {
233     name: "Croatian",
234     iso: "hrv",
235     writingDirection: "left-to-right",
236     scriptType: "alphabetic",
237 },
238 {
239     name: "Bulgarian",
240     iso: "bul",
241     writingDirection: "left-to-right",
242     scriptType: "alphabetic",
243 },
244 {
245     name: "Greek",
246     iso: "ell",
247     writingDirection: "left-to-right",
248     scriptType: "alphabetic",
249 },
250 {
251     name: "Turkish",
252     iso: "tur",
253     writingDirection: "left-to-right",
254     scriptType: "alphabetic",
255 },
256 {
257     name: "Finnish",
258     iso: "fin",
259     writingDirection: "left-to-right",
260     scriptType: "alphabetic",
261 },
262 {
263     name: "Swedish",
264     iso: "swe",
265     writingDirection: "left-to-right",
266     scriptType: "alphabetic",
267 },
268 {
269     name: "Norwegian",
270     iso: "nor",
271     writingDirection: "left-to-right",
272     scriptType: "alphabetic",
273 },
274 {
275     name: "Danish",
276     iso: "dan",
277     writingDirection: "left-to-right",
278     scriptType: "alphabetic",
279 },
280 {
281     name: "Icelandic",
282     iso: "isl",
283     writingDirection: "left-to-right",
284     scriptType: "alphabetic",
285 },
286 {
287     name: "Irish",
288     iso: "gle",
289     writingDirection: "left-to-right",
290     scriptType: "alphabetic",
291 },
292 {
293     name: "Welsh",
294     iso: "cym",
295     writingDirection: "left-to-right",
296     scriptType: "alphabetic",
297 },
298 {
299     name: "Arabic",
300     iso: "ara",
301     writingDirection: "right-to-left",
302     scriptType: "alphabetic",
303 },
304 {
305     name: "Hebrew",
306     iso: "heb",

```

```

307     writingDirection: "right-to-left",
308     scriptType: "alphabetic",
309 },
310 {
311     name: "Persian",
312     iso: "fas",
313     writingDirection: "right-to-left",
314     scriptType: "alphabetic",
315 },
316 {
317     name: "Urdu",
318     iso: "urd",
319     writingDirection: "right-to-left",
320     scriptType: "alphabetic",
321 },
322 {
323     name: "Hindi",
324     iso: "hin",
325     writingDirection: "left-to-right",
326     scriptType: "syllabic",
327 },
328 {
329     name: "Bengali",
330     iso: "ben",
331     writingDirection: "left-to-right",
332     scriptType: "syllabic",
333 },
334 {
335     name: "Tamil",
336     iso: "tam",
337     writingDirection: "left-to-right",
338     scriptType: "syllabic",
339 },
340 {
341     name: "Telugu",
342     iso: "tel",
343     writingDirection: "left-to-right",
344     scriptType: "syllabic",
345 },
346 {
347     name: "Thai",
348     iso: "tha",
349     writingDirection: "left-to-right",
350     scriptType: "syllabic",
351 },
352 {
353     name: "Lao",
354     iso: "lao",
355     writingDirection: "left-to-right",
356     scriptType: "syllabic",
357 },
358 {
359     name: "Chinese",
360     iso: "zho",
361     writingDirection: "left-to-right",
362     scriptType: "logographic",
363 },
364 {
365     name: "Japanese",
366     iso: "jpn",
367     writingDirection: "left-to-right",
368     scriptType: "logographic",
369 },
370 {
371     name: "Korean",
372     iso: "kor",
373     writingDirection: "left-to-right",
374     scriptType: "syllabic",
375 },
376 {
377     name: "Vietnamese",
378     iso: "vie",
379     writingDirection: "left-to-right",
380     scriptType: "alphabetic",
381 },
382 {
383     name: "Indonesian",
384     iso: "ind",

```

```

385     writingDirection: "left-to-right",
386     scriptType: "alphabetic",
387   },
388 ];
389
390 export const weights = [
391   "Thin",
392   "ExtraLight",
393   "Light",
394   "Regular",
395   "Medium",
396   "SemiBold",
397   "Bold",
398   "ExtraBold",
399   "Black",
400 ];
401
402 export const slopes = ["Upright", "Italic", "Oblique"];
403
404 export const alphabetStatuses = ["full", "partial", "unsupported"];
405
406 export const tables = [
407   "Category",
408   "Format",
409   "Language",
410   "Font_family",
411   "Typeface",
412   "Alphabet",
413 ];
414
415
416 // seed.ts
417 import { faker } from "@faker-js/faker";
418 import { pool } from "./db.js";
419 import {
420   BASE_COUNTS,
421   COEFFICIENTS,
422   COUNTS,
423   categoryNames,
424   categoryDescs,
425   formats,
426   languages,
427   weights,
428   slopes,
429   alphabetStatuses,
430   tables,
431 } from "./constants.js";
432 import type { PoolClient } from "pg";
433
434 type IdMap = Map<string, number>;
435
436 type TypefaceRow = {
437   id: number;
438   fontFamilyId: number;
439 };
440
441 class Seeder {
442   private client!: PoolClient;
443
444   private categoryIds: number[] = [];
445   private formatIds: number[] = [];
446   private languageIds: number[] = [];
447   private fontFamilyIds: number[] = [];
448   private typefaces: TypefaceRow[] = [];
449
450   private weightIds: IdMap = new Map();
451   private slopeIds: IdMap = new Map();
452   private extensionIds: IdMap = new Map();
453   private writingDirectionIds: IdMap = new Map();
454   private scriptTypeIds: IdMap = new Map();
455
456   async run() {
457     this.client = await pool.connect();
458
459     try {
460       await this.client.query("BEGIN");
461
462       await this.clearTables();

```



```

463
464     await this.seedDictionaries();
465
466     await this.seedCategories();
467     await this.seedFormats();
468     await this.seedLanguages();
469
470     await this.seedFontFamilies();
471     await this.seedTypefaces();
472     await this.seedAlphabets();
473
474     await this.updateAlphabetCounts();
475     await this.updateCategoryCounts();
476
477     await this.client.query("COMMIT");
478
479     console.log("\nSeeding complete!");
480     await this.printSummary();
481   } catch (error) {
482     await this.client.query("ROLLBACK");
483     throw error;
484   } finally {
485     this.client.release();
486     await pool.end();
487   }
488 }
489
490 private async clearTables() {
491   await this.client.query(`
492     TRUNCATE TABLE
493       Alphabet,
494       Typeface,
495       Font_family,
496       Language,
497       Format,
498       Category,
499       Weight,
500       Slope,
501       Extension,
502       Writing_direction,
503       Script_type
504   RESTART IDENTITY CASCADE
505 `);
506
507   console.log("Tables cleared");
508 }
509
510 private async seedDictionaries() {
511   await this.seedWeightDictionary();
512   await this.seedSlopeDictionary();
513   await this.seedExtensionDictionary();
514   await this.seedWritingDirectionDictionary();
515   await this.seedScriptTypeDictionary();
516 }
517
518 private async seedWeightDictionary() {
519   for (const name of weights) {
520     const result = await this.client.query(
521       `
522       INSERT INTO Weight (Name)
523       VALUES ($1)
524       RETURNING id_weight
525       `,
526       [name],
527     );
528
529     this.weightIds.set(name, result.rows[0].id_weight);
530   }
531
532   console.log(`Weight: ${this.weightIds.size}`);
533 }
534
535 private async seedSlopeDictionary() {
536   for (const name of slopes) {
537     const result = await this.client.query(
538       `
539       INSERT INTO Slope (Name)
540       VALUES ($1)

```

```

541         RETURNING id_Slope
542     `,
543     [name],
544 );
545
546     this.slopeIds.set(name, result.rows[0].id_slope);
547 }
548
549     console.log(`Slope: ${this.slopeIds.size}`);
550 }
551
552 private async seedExtensionDictionary() {
553     const uniqueExtensions = [...new Set(formats.map((format) => format.ext))];
554
555     for (const extension of uniqueExtensions) {
556         const result = await this.client.query(
557             `
558             INSERT INTO Extension (Name)
559             VALUES ($1)
560             RETURNING id_Extension
561             `,
562             [extension],
563         );
564
565         this.extensionIds.set(extension, result.rows[0].id_extension);
566     }
567
568     console.log(`Extension: ${this.extensionIds.size}`);
569 }
570
571 private async seedWritingDirectionDictionary() {
572     const selectedLanguages = languages.slice(0, BASE_COUNTS.languages);
573     const uniqueWritingDirections = [
574         ...new Set(
575             selectedLanguages.map((language) => language.writingDirection),
576         ),
577     ];
578
579     for (const name of uniqueWritingDirections) {
580         const result = await this.client.query(
581             `
582             INSERT INTO Writing_direction (Name)
583             VALUES ($1)
584             RETURNING id_Writing_direction
585             `,
586             [name],
587         );
588
589         this.writingDirectionIds.set(name, result.rows[0].id_writing_direction);
590     }
591
592     console.log(`Writing_direction: ${this.writingDirectionIds.size}`);
593 }
594
595 private async seedScriptTypeDictionary() {
596     const selectedLanguages = languages.slice(0, BASE_COUNTS.languages);
597     const uniqueScriptTypes = [
598         ...new Set(selectedLanguages.map((language) => language.scriptType)),
599     ];
600
601     for (const name of uniqueScriptTypes) {
602         const result = await this.client.query(
603             `
604             INSERT INTO Script_type (Name)
605             VALUES ($1)
606             RETURNING id_Script_type
607             `,
608             [name],
609         );
610
611         this.scriptTypeIds.set(name, result.rows[0].id_script_type);
612     }
613
614     console.log(`Script_type: ${this.scriptTypeIds.size}`);
615 }
616
617 private async seedCategories() {
618     const selectedCategories = categoryNames.slice(0, BASE_COUNTS.categories);

```

```

619
620   for (const name of selectedCategories) {
621     const result = await this.client.query(
622       `
623         INSERT INTO Category
624         (Name, Description, Icon, Font_family_count)
625         VALUES
626         ($1, $2, $3, 0)
627         RETURNING id_Category
628       `,
629       [name, categoryDescs[name], `/${icons}/${this.slugify(name)}.svg`],
630     );
631
632     this.categoryIds.push(result.rows[0].id_category);
633   }
634
635   console.log(`Category: ${this.categoryIds.length}`);
636 }
637
638 private async seedFormats() {
639   const selectedFormats = formats.slice(0, BASE_COUNTS.formats);
640
641   for (const format of selectedFormats) {
642     const extensionId = this.getRequiredId(
643       this.extensionIds,
644       format.ext,
645       "Extension",
646     );
647
648     const result = await this.client.query(
649       `
650         INSERT INTO Format
651         (Name, id_Extension, Year_released, Description, Web_support)
652         VALUES
653         ($1, $2, $3, $4, $5)
654         RETURNING id_Format
655       `,
656       [format.name, extensionId, format.year, format.desc, format.web],
657     );
658
659     this.formatIds.push(result.rows[0].id_format);
660   }
661
662   console.log(`Format: ${this.formatIds.length}`);
663 }
664
665 private async seedLanguages() {
666   const selectedLanguages = languages.slice(0, BASE_COUNTS.languages);
667
668   for (const language of selectedLanguages) {
669     const writingDirectionId = this.getRequiredId(
670       this.writingDirectionIds,
671       language.writingDirection,
672       "Writing_direction",
673     );
674
675     const scriptTypeId = this.getRequiredId(
676       this.scriptTypeIds,
677       language.scriptType,
678       "Script_type",
679     );
680
681     const result = await this.client.query(
682       `
683         INSERT INTO Language
684         (
685           Name,
686           ISO_code,
687           id_Writing_direction,
688           id_Script_type
689         )
690         VALUES
691         ($1, $2, $3, $4)
692         RETURNING id_Language
693       `,
694       [language.name, language.iso, writingDirectionId, scriptTypeId],
695     );
696

```

```

697     this.languageIds.push(result.rows[0].id_language);
698 }
699
700 console.log(`Language: _${this.languageIds.length}_`);
701 }
702
703 private async seedFontFamilies() {
704     let created = 0;
705
706     const batchSize = 1000;
707     let values: unknown[] = [];
708     let placeholders: string[] = [];
709
710     for (const categoryId of this.categoryIds) {
711         for (let i = 0; i < COEFFICIENTS.fontFamiliesPerCategory; i++) {
712             const index = values.length;
713
714             placeholders.push(
715                 `($${index + 1}, $${index + 2}, $${index + 3}, $${index + 4}, $${index + 5})`,
716             );
717
718             values.push(
719                 this.makeFontFamilyName(created),
720                 faker.number.int({ min: 1950, max: 2024 }),
721                 faker.lorem.sentence(),
722                 COEFFICIENTS.typefacesPerFamily,
723                 categoryId,
724             );
725
726             created++;
727
728             if (placeholders.length >= batchSize) {
729                 await this.insertFontFamilyBatch(placeholders, values);
730                 values = [];
731                 placeholders = [];
732             }
733         }
734
735         console.log(
736             `Font_family by category: _${this.categoryIds.indexOf(categoryId) + 1}_/$${this.categoryIds.length}_,
737         );
738     }
739
740     if (placeholders.length > 0) {
741         await this.insertFontFamilyBatch(placeholders, values);
742     }
743
744     console.log(`Font_family: _${this.fontFamilyIds.length}_`);
745 }
746
747 private async insertFontFamilyBatch(
748     placeholders: string[],
749     values: unknown[],
750 ) {
751     const result = await this.client.query(
752         `
753         INSERT INTO Font_family
754         (
755             Name,
756             Year_created,
757             Description,
758             Typeface_count,
759             id_Category
760         )
761         VALUES
762             ${placeholders.join(", ")}
763         RETURNING id_Font_family
764         `,
765         values,
766     );
767
768     for (const row of result.rows) {
769         this.fontFamilyIds.push(row.id_font_family);
770     }
771 }
772

```

```

773 private async seedTypefaces() {
774     let created = 0;
775
776     const batchSize = 1000;
777     let values: unknown[] = [];
778     let placeholders: string[] = [];
779
780     for (const fontFamilyId of this.fontFamilyIds) {
781         const familyName = this.makeTypefaceFamilyPrefix(fontFamilyId);
782
783         for (let i = 0; i < COEFFICIENTS.typefacesPerFamily; i++) {
784             const weight = weights[i % weights.length];
785             const slope = slopes[i % slopes.length];
786
787             const weightId = this.getRequiredId(this.weightIds, weight, "Weight");
788             const slopeId = this.getRequiredId(this.slopeIds, slope, "Slope");
789
790             const formatId = this.formatIds[i % this.formatIds.length];
791
792             const typefaceName =
793                 slope === "Upright"
794                     ? `${familyName} ${weight}`
795                     : `${familyName} ${weight} ${slope}`;
796
797             const index = values.length;
798
799             placeholders.push(
800                 `($${index + 1}, $${index + 2}, $${index + 3}, $${index + 4}, $${index + 5}, $${index + 6})`,
801             );
802
803             values.push(
804                 typefaceName.slice(0, 100),
805                 weightId,
806                 slopeId,
807                 faker.number.int({ min: 30_000, max: 400_000 }),
808                 fontFamilyId,
809                 formatId,
810             );
811
812             created++;
813
814             if (placeholders.length >= batchSize) {
815                 await this.insertTypefaceBatch(placeholders, values);
816                 values = [];
817                 placeholders = [];
818
819                 if (created % 5000 === 0) {
820                     console.log(`Typeface: ${created}/${COUNTS.typefaces}`);
821                 }
822             }
823         }
824     }
825
826     if (placeholders.length > 0) {
827         await this.insertTypefaceBatch(placeholders, values);
828     }
829
830     console.log(`Typeface: ${this.typefaces.length}`);
831 }
832
833 private async insertTypefaceBatch(placeholders: string[], values: unknown[]) {
834     const result = await this.client.query(
835         `
836         INSERT INTO Typeface
837         (
838             Name,
839             id_Weight,
840             id_Slope,
841             File_size,
842             id_Font_family,
843             id_Format
844         )
845         VALUES
846             ${placeholders.join(", ")}
847         RETURNING id_Typeface, id_Font_family
848     `,
849     values,

```

```

850     );
851
852     for (const row of result.rows) {
853         this.typefaces.push({
854             id: row.id_typeface,
855             fontFamilyId: row.id_font_family,
856         });
857     }
858 }
859
860 private async seedAlphabets() {
861     let created = 0;
862
863     const alphabetPairs = this.buildAlphabetPairs();
864
865     const batchSize = 5000;
866     let values: unknown[] = [];
867     let placeholders: string[] = [];
868
869     for (const [typefaceId, languageId] of alphabetPairs) {
870         const index = values.length;
871
872         placeholders.push(
873             `(${index + 1}, ${index + 2}, ${index + 3}, ${index + 4}, ${index + 5}, ${index + 6})`,
874         );
875
876         values.push(
877             0,
878             0,
879             this.makeVerificationDate(),
880             this.makeAlphabetStatus(),
881             typefaceId,
882             languageId,
883         );
884
885         created++;
886
887         if (placeholders.length >= batchSize) {
888             await this.insertAlphabetBatch(placeholders, values);
889             values = [];
890             placeholders = [];
891
892             if (created % 50_000 === 0) {
893                 console.log(` Alphabet: ${created}/${alphabetPairs.length}`);
894             }
895         }
896     }
897
898     if (placeholders.length > 0) {
899         await this.insertAlphabetBatch(placeholders, values);
900     }
901
902     console.log(` Alphabet: ${created}`);
903 }
904
905 private buildAlphabetPairs(): Array<number, number> {
906     const languagesPerTypeface = COEFFICIENTS.alphabetsPerTypeface;
907     const typefacesPerLanguage = COEFFICIENTS.alphabetsPerLanguage;
908
909     if (this.typefaces.length === 0) {
910         throw new Error("Cannot seed Alphabet: no typefaces");
911     }
912
913     if (this.languageIds.length === 0) {
914         throw new Error("Cannot seed Alphabet: no languages");
915     }
916
917     if (languagesPerTypeface > this.languageIds.length) {
918         throw new Error(
919             `Cannot seed Alphabet: alphabetsPerTypeface=${languagesPerTypeface}, but
920             ↪ languages=${this.languageIds.length}`,
921         );
922     }
923
924     if (typefacesPerLanguage > this.typefaces.length) {
925         throw new Error(

```

```

925     `Cannot seed Alphabet: alphabetsPerLanguage=${typefacesPerLanguage}, but
926     ↪ typefaces=${this.typefaces.length}`,
927   );
928 }
929
930 const totalAlphabetRows =
931   this.typefaces.length * languagesPerTypeface +
932   this.languageIds.length * typefacesPerLanguage;
933
934 if (totalAlphabetRows !== COUNTS.alphabets) {
935   console.warn(
936     `Alphabet target mismatch: generated ${totalAlphabetRows}, constants contain ${COUNTS.alphabets}`,
937   );
938 }
939
940 if (totalAlphabetRows > this.typefaces.length * this.languageIds.length) {
941   throw new Error(
942     `Cannot seed Alphabet: requested ${totalAlphabetRows} unique pairs, but maximum is ${
943       this.typefaces.length * this.languageIds.length
944     }`,
945   );
946 }
947
948 const pairs: Array<[number, number]> = [];
949 const usedPairs = new Set<string>();
950
951 const addPair = (typefaceId: number, languageId: number) => {
952   const key = `${typefaceId}:${languageId}`;
953
954   if (usedPairs.has(key)) {
955     throw new Error(
956       `Cannot seed Alphabet: duplicated pair typeface=${typefaceId}, language=${languageId}`,
957     );
958   }
959
960   usedPairs.add(key);
961   pairs.push([typefaceId, languageId]);
962 }
963
964 for (
965   let typefaceIndex = 0;
966   typefaceIndex < this.typefaces.length;
967   typefaceIndex++
968 ) {
969   const typeface = this.typefaces[typefaceIndex];
970
971   for (let offset = 0; offset < languagesPerTypeface; offset++) {
972     const languageIndex =
973       (typefaceIndex + offset) % this.languageIds.length;
974
975     const languageId = this.languageIds[languageIndex];
976
977     addPair(typeface.id, languageId);
978   }
979 }
980
981 for (
982   let languageIndex = 0;
983   languageIndex < this.languageIds.length;
984   languageIndex++
985 ) {
986   const languageId = this.languageIds[languageIndex];
987   const availableTypefaces = this.typefaces.filter(
988     (typeface) => !usedPairs.has(`${typeface.id}:${languageId}`),
989   );
990
991   if (typefacesPerLanguage > availableTypefaces.length) {
992     throw new Error(
993       `Cannot seed Alphabet: language ${languageId} has only ${availableTypefaces.length} available
994       ↪ typefaces`,
995     );
996   }
997
998   for (let offset = 0; offset < typefacesPerLanguage; offset++) {
999     const typefaceIndex =
1000       (languageIndex + offset) % availableTypefaces.length;
1001     const typeface = availableTypefaces[typefaceIndex];

```

```

1000     addPair(typeface.id, languageId);
1001   }
1002 }
1003
1004
1005 if (pairs.length !== totalAlphabetRows) {
1006   throw new Error(
1007     `Cannot seed Alphabet: created ${pairs.length}/${totalAlphabetRows} rows`,
1008   );
1009 }
1010
1011 return pairs;
1012 }
1013
1014 private async insertAlphabetBatch(placeholders: string[], values: unknown[]) {
1015   await this.client.query(
1016     `
1017     INSERT INTO Alphabet
1018     (
1019       Typeface_count_in_system,
1020       Supported_family_count,
1021       Verification_date,
1022       Status,
1023       id_Typeface,
1024       id_Language
1025     )
1026     VALUES
1027       ${placeholders.join(", ")}
1028     `,
1029     values,
1030   );
1031 }
1032
1033 private async updateAlphabetCounts() {
1034   await this.client.query(`
1035     UPDATE Alphabet a
1036     SET
1037       Typeface_count_in_system = stats.typeface_count,
1038       Supported_family_count = stats.family_count
1039     FROM (
1040       SELECT
1041         a2.id_Language,
1042         COUNT(DISTINCT a2.id_Typeface) AS typeface_count,
1043         COUNT(DISTINCT t.id_Font_family) AS family_count
1044       FROM Alphabet a2
1045       JOIN Typeface t ON t.id_Typeface = a2.id_Typeface
1046       GROUP BY a2.id_Language
1047     ) stats
1048     WHERE stats.id_Language = a.id_Language
1049   `);
1050
1051   console.log("Alphabet counts updated");
1052 }
1053
1054 private async updateCategoryCounts() {
1055   await this.client.query(`
1056     UPDATE Category
1057     SET Font_family_count = (
1058       SELECT COUNT(*)
1059       FROM Font_family
1060       WHERE Font_family.id_Category = Category.id_Category
1061     )
1062   `);
1063
1064   console.log("Category counts updated");
1065 }
1066
1067 private makeFontFamilyName(index: number): string {
1068   const adjective = faker.word.adjective();
1069   const noun = faker.word.noun();
1070
1071   const generated = `${adjective} ${noun}`
1072     .split(" ")
1073     .map((part) => part.charAt(0).toUpperCase() + part.slice(1))
1074     .join(" ");
1075
1076   return `${generated} ${index + 1}`.slice(0, 100);
1077 }

```



```

1078
1079 private makeTypefaceFamilyPrefix(fontFamilyId: number): string {
1080     return `Family ${fontFamilyId}`;
1081 }
1082
1083 private makeVerificationDate(): string | null {
1084     return (
1085         faker.helpers.maybe(
1086             () => faker.date.past({ years: 2 }).toISOString().split("T")[0],
1087             { probability: 0.7 },
1088         ) ?? null
1089     );
1090 }
1091
1092 private makeAlphabetStatus(): string {
1093     return faker.helpers.arrayElement(alphabetStatuses);
1094 }
1095
1096 private getRequiredId(map: IdMap, key: string, tableName: string): number {
1097     const id = map.get(key);
1098
1099     if (id === undefined) {
1100         throw new Error(`Missing dictionary value in ${tableName}: ${key}`);
1101     }
1102
1103     return id;
1104 }
1105
1106 private slugify(value: string): string {
1107     return value.toLowerCase().replaceAll(" ", "-").replaceAll("_", "-");
1108 }
1109
1110 private async printSummary() {
1111     let total = 0;
1112
1113     console.log("\nTable counts:");
1114
1115     for (const table of tables) {
1116         const result = await pool.query(`SELECT COUNT(*) FROM ${table}`);
1117         const count = Number(result.rows[0].count);
1118
1119         total += count;
1120         console.log(` ${table}: ${count}`);
1121     }
1122
1123     console.log(` TOTAL: ${total}`);
1124 }
1125 }
1126
1127 new Seeder().run().catch((error) => {
1128     console.error(error);
1129     process.exit(1);
1130 });
1131
1132
1133

```